
SK Networks Family AI 27기: 4팀
멀티 에이전트 테스트 계획 및 결과 보고서

산출물 단계 모델링 및 평가

제출 일자 2026.07.27

깃허브 경로 [https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN27-FI
NAL-4Team](https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN27-FINAL-4Team)

작성 팀원 한재웅

1. 문서 개요

본 보고서는 SK Networks Family AI Camp 27기 4팀의 마음리포트 백엔드에 구현된 멀티에이전트가 설계한 순서와 조건에 따라 동작하는지 검증한 결과를 정리한 제출 문서이다.

검증 범위는 에이전트 단계 전환, 생성형 AI 연결, KcELECTRA 감정 분류, PostgreSQL 원문·리포트 저장, Neo4j 장기기억 조회, Tavily 외부 검색, API·예약 실행·안전 응답, 기능별 소스 분리이다.

1.1 검증 원칙

- 테스트 수행 주체는 프로젝트 팀이며, LLM과 KcELECTRA는 테스트 대상 시스템의 구성요소이다.
- 각 테스트는 입력·절차·합격 기준을 먼저 정한 뒤, 프로그램 종료 상태, 자동 검증 결과, 응답 형식, 데이터베이스 조회값을 근거로 판정하였다.
- 외부 서비스 자체의 상태와 프로젝트의 연동 기능을 구분했으며, 반복 검사가 필요한 경우에는 개인정보를 제거한 테스트 데이터와 미리 정한 응답을 사용하였다.
- 테스트용 사용자·리포트·그래프 노드는 실행 후 삭제하여 저장소에 시험 데이터가 남지 않도록 확인하였다.

2. 마음리포트 백엔드 구현 분석

2.1 멀티에이전트 실행 구조

- 마음리포트의 전체 흐름을 관리하는 감독 에이전트(MindReportSupervisorAgent)는 8개 업무 단계를 조건에 따라 연결한다. 실제 실행 구조는 시작·종료를 포함한 10개 단계와 단계 사이의 연결 19개로 확인되었다.

- 대화 수집과 생성 기준 확인(collect_and_check_criteria): 주간 5건·월간 20건 기준을 확인하고 PostgreSQL 원문과 Neo4j 장기기억을 가져온다.
- 감정 분석(score_and_analyze_emotion): KcELECTRA가 계산한 감정별 확률을 날짜별로 모아 감정 점수와 변화 흐름을 계산한다.
- 원인 분석(extract_and_classify_causes): 대화 원문에 근거한 핵심 표현을 찾아 스트레스 요인·완화 요인·미해결 요인으로 나눈다.
- 분석 문장과 실천 제안 생성(generate_narrative_and_actions): 확인된 원인과 장기기억 맥락을 바탕으로 설명 문장과 실천 카드를 만든다.
- 리포트 검증(validate_report): 사용한 원문과 장기기억 사건이 실제로 존재하는지, 분석 기간·출력 형식·안전 표현이 올바른지 확인하고 필요하면 문제가 생긴 단계만 다시 실행한다.
- 최종 응답 구성(format_report·fallback_report·safety_response): 정상 리포트, 데이터 부족 안내, 고위험 안전 응답을 화면과 API에서 사용할 최종 형식으로 정리한다.

2.2 모델과 외부 서비스의 역할

- KcELECTRA 4-class 모델: 입력 문장을 기쁨·슬픔·분노·일반의 네 감정으로 분류하도록 학습된 모델이다. 각 문장에 대해 네 감정의 확률을 계산하며, 이 값은 날짜별 감정 점수 산정에 사용된다.
- OpenAI 생성형 LLM: 대화 원문에서 핵심 표현을 찾고, 감정 원인을 분류하며, 분석 문장과 실천 카드를 만드는 데 사용된다. 결과는 정해진 JSON 형식으로 받고, 실제로 제공한 원문과 장기기억 사건의 식별번호만 근거로 인정한다.
- Tavily Search API: 정규 리포트를 만들 대화가 부족할 때만 가벼운 활동 정보를 검색한다. 검색 결과에서 제목·주소·본문이 확인된 경우에만 OpenAI가 이를 근거로 제안을 만들며, 미리 작성된 고정 추천은 사용하지 않는다.

2.3 PostgreSQL 원문과 Neo4j 장기기억의 결합

- 마음리포트의 가장 중요한 근거는 PostgreSQL에 저장된 사용자 대화 원문이다. 원문 수집 기능(load_source_messages)은 분석 기간 안에서 사용자가 작성한 메시지만 시간순으로 가져오고, 원문 식별번호·작성일·내용·저장된 감정 라벨을 리포트 분석용 데이터로 정리한다.
- Neo4j는 대화에서 구조화해 둔 장기기억 사건을 제공한다. 장기기억 수집 기능(collect_ltm_events)은 같은 사용자의 사건을 분석 기간으로 조회하고, 사건 날짜·관련 인물·주제·감정·장소 정보를 함께 가져온다.
- 핵심 표현을 찾는 단계에는 대화 원문과 구조화된 장기기억 사건이 함께 전달된다. 모든 표현은 반드시 실제 원문을 근거로 해야 하며, 원문과 연결이 확인된 사건만 보조 맥락으로 사용한다. 이후 원인 분석과 설명 문장 생성에는 확인된 근거만 전달되고, 검증 단계에서 존재하지 않는 식별번호와 분석 기간 밖의 날짜를 제외한다.

3. 사전 테스트 계획과 합격 기준

TP-01 멀티에이전트 라우팅과 상태 전이

- 검증 목적: 감독 에이전트가 입력 상태에 따라 정상 리포트 생성, 데이터 부족 안내, 오류 단계 재처리, 고위험 안전 응답 중 올바른 흐름을 선택하는지 확인한다.
- 입력·사전 조건: 주간 5건·월간 20건 기준을 충족한 테스트 데이터와 기준에 미달한 데이터, 원인 분석·설명 문장 검증 오류, 최근 24시간 내 고위험 표현을 각각 준비한다.
- 실행·관측 항목: 에이전트의 실행 순서, 다음 단계, 검증 결과, 다시 실행할 단계, 재시도 횟수와 최종 응답을 비교해 의도한 단계로 이동했는지 확인한다.
- 합격 기준: 정상 입력은 기준 확인→감정 분석→원인 분석→설명·실천 제안→검증→최종 형식화 순서로 처리되어야 한다. 데이터가 부족하면 안내 응답으로, 고위험 입력은 안전 응답으로 끝나야 하며, 원인 분석이나 설명 생성 오류는 최대 1회만 해당 단계에서 다시 처리되어야 한다.

TP-02 KcELECTRA 지도학습 감정 분류

- 검증 목적: 학습된 KcELECTRA 감정 분류 모델이 마음리포트 계산에 필요한 네 감정의 확률을 올바른 형식으로 제공하는지 확인한다.
- 입력·사전 조건: 개인정보가 없는 한국어 문장 3개와 로컬 모델 파일을 준비하고, CPU 환경에서 감정 순서를 기쁨·슬픔·분노·일반으로 고정한다.
- 실행·관측 항목: 모델이 정상적으로 준비되었는지와 CPU 사용 여부를 확인한 뒤 문장 3개를 한 번에 분류한다. 문장별 네 감정 확률, 비정상 수치 여부, 확률합, 감정 순서와 소요 시간을 기록한다.
- 합격 기준: 모델을 사용할 수 있어야 하고, 결과는 문장 3개×감정 4종의 크기여야 한다. 모든 확률은 정상 수치이고 문장별 합계가 허용 오차 내 1.0이어야 하며, 감정 순서는 기쁨·슬픔·분노·일반으로 유지되어야 한다.

TP-03 원문과 Neo4j 장기기억 수집

- 검증 목적: PostgreSQL의 대화 원문과 Neo4j의 장기기억 사건이 같은 사용자와 분석 기간을 기준으로 수집되어 다음 분석 단계의 근거로 전달되는지 확인한다. 장기기억은 유사 문서를 찾는 방식이 아니라 사용자 식별값과 기간이 일치하는 사건을 직접 조회한다.
- 입력·사전 조건: 대상 기간에 사용자가 작성한 대화 원문과 다른 데이터와 구분되는 임시 사용자 식별값을 준비한다. Neo4j에는 사용자·에피소드·사건과 관련 인물·감정·장소·주제 정보를 가진 시험 데이터를 저장한다.
- 실행·관측 항목: 원문의 식별번호·내용·작성일·감정 라벨이 그대로 유지되는지 확인한다. 이어 실제 장기기억 조회 기능(collect_ltm_events)으로 사건을 다시 가져와 사건 식별번호와 관련 인물·감정·장소·주제가 분석용 장기기억 형식에 들어가는지 검사한다.
- 합격 기준: 대상 사용자와 기간에 해당하는 원문만 시간순으로 수집되고, 시험 사건 1건과 관련 정보가 빠짐없이 전달되어야 한다. 다른 사용자나 기간의 데이터가 섞이지 않아야 하며, 검사 후 Neo4j에 시험 데이터가 남지 않아야 한다.

TP-04 OpenAI 생성형 LLM 연동

- 검증 목적: OpenAI API 연결, 정해진 JSON 응답 형식, 필수 항목 처리와 원문·장기기억 근거 제한이 정상적으로 작동하는지 확인한다.
- 입력·사전 조건: 개인정보를 제거한 근거 데이터 1건과 사용을 허용한 원문·사건 식별번호 목록을 준비한다. 실제 OpenAI API 검사는 웹 근거 추천 1회로 제한하고, 핵심 표현·원인·설명 생성 단계는 결과를 미리 정할 수 있는 모의 응답을 함께 사용한다.
- 실행·관측 항목: API 응답 성공 여부와 소요 시간, JSON 형식 해석, 추천 결과 수, 활동명·추천 이유·실천 방법의 포함 여부를 기록한다. 모의 응답에는 허용 목록 밖의 식별번호도 넣어 결과에서 자동으로 제외되는지 확인한다.
- 합격 기준: 실제 응답이 JSON 형식으로 해석되고 활동명·추천 이유·실천 방법을 갖춘 추천이 1건 이상 생성되어야 한다. 핵심 표현·원인·설명 결과에는 허용된 근거 식별번호만 남아야 하며, 응답 형식이 다르거나 해석에 실패하면 합격으로 판정하지 않는다.

TP-05 PostgreSQL 리포트 연속성

- 검증 목적: 생성된 마음리포트가 실제 PostgreSQL 데이터베이스에 저장되고, 같은 기간의 리포트 갱신·중복 방지·데이터 부족 안내 교체·조회 규칙이 설계대로 작동하는지 확인한다.
- 입력·사전 조건: 기존 데이터와 구분되는 임시 시험 사용자, 주간 분석 기간, 데이터 부족 안내 리포트와 정상 리포트를 준비한다.
- 실행·관측 항목: PostgreSQL 연결, 안전한 매개변수 방식의 저장·조회, 최초 저장, 같은 기간 갱신, 중복 제거, JSON 저장값 재확인, 데이터 부족 안내를 정상 리포트로 교체, 기간별 존재 여부와 최신 리포트 조회를 순서대로 검사한다.
- 합격 기준: 9개 검사 항목이 모두 통과하고, 같은 기간을 갱신할 때 기존 저장 항목을 재사용해 최종 리포트가 1건만 남아야 한다. 저장 전후 JSON 내용이 같고 데이터 부족 표시가 정상 리포트로 바뀌어야 하며, 검사 후 임시 사용자와 리포트가 남지 않아야 한다.

TP-06 Tavily 외부 검색 연동 모듈

- 검증 목적: 대화 데이터가 부족할 때 사용하는 Tavily 연동 모듈이 정해진 검색 조건을 지키고, 검색 결과를 추천 생성 단계에 정확히 한 번 전달하는지 확인한다.

- 입력·사전 조건: 본문이 있는 검색 결과 1건과 본문이 비어 있는 결과 1건을 포함한 정상 응답을 미리 준비한다. 외부 서비스의 일시적 상태와 관계없이 프로젝트의 연동 기능을 반복 검사할 수 있도록 응답 내용을 고정한다.
- 실행·관측 항목: **Tavily API** 주소, 검색어, 기본 검색 단계, 최대 결과 3건, 응답 대기 5초 설정을 확인한다. 검색 결과의 제목·주소·본문 정리와 본문 1,500자 제한을 검사하고, 검색 결과가 추천 생성으로 몇 번 전달되는지 확인한다.
- 합격 기준: **Tavily** 요청·결과 처리 검사 6개와 검색 결과 전달 흐름 검사 4개가 모두 통과해야 한다. 본문이 없는 결과는 제외되고, 유효한 검색 결과가 검색 1회와 추천 생성 1회의 흐름으로 전달되어 활동명이 포함된 추천 1건이 반환되어야 한다.

TP-07 API·스케줄러·안전 응답

- 검증 목적: 마음리포트 **API**의 로그인 확인·조회·생성·갱신 규칙, 주간·월간 분석 기간 계산, 데이터 부족·안전 응답, 예약 생성 흐름을 전체 자동 테스트로 확인한다.
- 입력·사전 조건: 로그인 여부가 다른 요청, 저장된 리포트의 존재 여부, 주간·월간 경계 날짜, 새로고침 요청, 최근 24시간 전후의 고위험 신호와 예약 실행 대상 사용자 데이터를 준비한다.
- 실행·관측 항목: **Django** 테스트 데이터베이스에서 조회·생성 요청의 응답 상태와 내용, 리포트 생성 호출 횟수, 분석 기간, 같은 기간 갱신, 데이터 부족 응답, 24시간 안전 조건, 주간 월요일·월간 1일 예약 실행과 사용자별 처리 여부를 자동으로 검사한다.
- 합격 기준: 준비된 78개 테스트가 모두 실행되고 실패와 오류가 없어야 한다. 비로그인 접근 차단, 직전에 완료된 주·월 계산, 같은 기간 중복 방지, 24시간 안전 기준과 사용자별 예약 실행 조건이 모두 사전에 정한 값과 일치해야 한다.

TP-08 모듈화와 그래프 구조

- 검증 목적: 마음리포트 백엔드가 실행 상태, 에이전트 흐름, 데이터 수집, 감정 계산, 원인·설명 생성, 검증, 최종 응답, 저장, 실행 환경 기능으로 나뉘어 있으며 실제로 함께 실행되는지 확인한다.
- 입력·사전 조건: 서비스 폴더의 24개 기능 모듈과 마음리포트 패키지, 실행 가능한 상태로 구성한 **LangGraph**를 사용한다. 단순 파일 수가 아니라 각 모듈을 실제로 불러올 수 있는지와 에이전트 흐름 구성을 판정 대상으로 삼는다.
- 실행·관측 항목: 패키지의 문법 오류를 먼저 확인한 뒤 서비스 모듈 24개를 하나씩 불러온다. 에이전트 실행 단계 수와 단계 사이 연결 수를 조회하고, 불러오기 오류·서로 반복 참조하는 문제·기능별 역할 분리가 없는지 확인한다.
- 합격 기준: 서비스 모듈 24개를 모두 오류 없이 불러올 수 있고, 에이전트 흐름은 10개 단계와 19개 연결로 구성되어야 한다. 8개 업무 단계는 수집·기준 확인, 감정 계산, 원인 분석, 설명·실천 제안, 검증, 정상 리포트 구성, 데이터 부족 안내, 안전 응답으로 나뉘어야 한다.

4. 테스트 실행 환경과 증거 수집 방법

4.1 실행 환경

- 소스 기준: **feature-mypage** 브랜치의 **a92207da** 커밋과 테스트 당시의 추가 변경 사항을 포함한 코드를 대상으로 하였다. 문서의 관측값은 이 코드와 설정을 기준으로 기록하였다.
- 백엔드 실행 환경: **wellness_django** 컨테이너에서 **Python 3.11.15**, **Django 4.2.30**, **DRF 3.17.1**, **LangChain 1.3.14**, **LangGraph 1.2.9**를 사용하였다.
- 모델 환경: **torch 2.13.0+cpu**와 로컬 **KcELECTRA** 모델 파일을 사용해 **CPU**에서 감정을 분류했으며, **GPU 가속(CUDA)**은 사용하지 않았다. **OpenAI**와 **Tavily** 인증 정보는 설정 여부만 확인하고 실제 키 값은 로그와 문서에 남기지 않았다.
- 저장소 환경: **PostgreSQL 15.18**의 **wellness_db**와 **Neo4j 5.26.28**을 사용하였다. 전체 **Django** 자동 테스트는 별도 테스트 데이터베이스에서 실행하고, 실제 저장소 검사는 임시 사용자 식별값과 시험 데이터 표시(**_codex_probe**)가 있는 데이터만 사용하였다.
- 격리·정리 조건: 입력값은 개인정보를 제거한 테스트 데이터로 구성하였다. **PostgreSQL**과 **Neo4j**에 넣은 시험 데이터는 성공·실패와 관계없이 정리되도록 했으며, 남은 데이터가 0건임을 확인해야 테스트가 완료된 것으로 보았다.

4.2 실행 명령

명령은 테스트 계획별로 나누어 실행했으며, 실행 화면에 테스트 수·자동 검증 결과·실측값·소요 시간을 함께 기록하였다. 명령 안에서 사용하는 시험 입력과 기대값은 3장의 사전 계획에 미리 정해 결과를 본 뒤 기준을 바꾸지 않도록 하였다.

- TP-01·TP-07 | `docker compose exec -T web python manage.py test mindreport -v 1` — 전체 78개 자동 테스트의 실행 수·성공·실패·오류와 에이전트 흐름·API 검증 결과 기록
- TP-03 | `docker compose exec -T web python manage.py test mindreport.tests.MindReportV2GraphCollectionTests -v 2` — 장기기억 전용 테스트 3개의 이름·개별 결과·소요 시간 기록
- TP-03·TP-05 | `docker compose exec -T web python manage.py verify_mindreport_storage_integration` — PostgreSQL 검사 9개와 Neo4j 실제 저장·조회·정리 후 잔여 건수 기록
- TP-02 | `docker compose exec -T web python manage.py shell -c <KcELECTRA 3문장 분류 검사 코드>` — 모델 사용 가능 여부·실행 장치·감정 순서·출력 크기·비정상 수치·확률합·소요 시간 기록
- TP-04 | `docker compose exec -T web python manage.py shell -c <OpenAI 응답 형식 검사 코드>` — JSON 응답·필수 항목·추천 결과 수·소요 시간 기록
- TP-06 | `docker compose exec -T web python manage.py shell -c <Tavily 요청·전달 흐름 검사 코드>` — 검색 요청 조건 6개와 검색 결과 전달 흐름 4개 검사 결과 기록
- TP-08 | `docker compose exec -T web python manage.py shell -c <24개 서비스 모듈·LangGraph 구조 검사 코드>` — 모듈 불러오기 성공 수와 실행 단계·연결·업무 단계 목록 기록

4.3 판정 근거 기록 방식

테스트 수행과 판정은 프로젝트 팀이 사전 계획에 따라 진행하였다. OpenAI 생성형 LLM과 KcELECTRA는 테스트 대상 구성요소이며, 모델이 작성한 자연어 설명이나 LLM의 주관적 평가는 PASS 근거로 사용하지 않았다.

- 사전 정의와 추적성: 실행 전에 각 테스트 계획의 목적, 입력·사전 조건, 실행 절차, 관측 항목, 합격 기준을 정하고, 5장의 결과 표에서 같은 번호로 실행 방법·실측값·판정을 연결하였다.
- 자동 판정 증거: Django 테스트 실행기의 종료 상태와 발견·실행·성공·실패·오류 건수, Python 자동 검사의 기대값과 실측값을 1차 근거로 사용하였다. 하나라도 다르거나 실행 오류가 발생하면 해당 테스트를 합격으로 판정하지 않는다.
- 모델·생성형 LLM 증거: KcELECTRA는 모델 사용 가능 여부, CPU 사용, 감정 순서, 출력 크기, 비정상 수치, 문장별 확률합을 수치로 확인하였다. OpenAI는 API 응답, JSON 형식 해석, 필수 항목, 허용된 근거 식별번호만 남는지를 확인하였다. 문장에 대한 주관적 인상은 합격 근거로 사용하지 않았다.
- 외부 API 증거: Tavily 연동 모듈은 미리 준비한 정상 응답으로 검색 조건과 결과 정리 기능을 반복 검사하였다. 검색 결과가 추천 생성으로 전달되는 과정은 각 기능의 호출 횟수와 전달된 값으로 확인해, 외부 서비스의 일시적 상태와 프로젝트 코드의 동작을 구분하였다.
- 저장소 증거: PostgreSQL은 저장 항목 식별번호·개수·JSON 재조화값·데이터 부족 표시·기간 조화 결과를 기록하였다. Neo4j는 실제 조화 결과의 사건 수·관련 정보·노드 수를 검사 항목별로 기록하였다. 고유한 시험 식별값과 실행 전후 건수 비교로 기존 데이터와 구분하였다.
- 정리와 재현성: 저장소 시험 데이터는 검사 성공 여부와 관계없이 삭제하고 잔여 건수 0을 확인하였다. 환경 버전·Git 기준·실행 명령·소요 시간을 함께 기록해 같은 조건에서 다시 실행할 수 있도록 했으며, 합격 기준은 결과 확인 후 바꾸지 않았다.

5. 실제 실행 결과와 도출 근거

테스트 케이스	실행 방법	핵심 관측값	판정
TP-01 라우팅·상태 전이	정상·기준 미달·오류 단계 재처리·안전 응답용 테스트 데이터를 전체 Django 자동 테스트로 실행	정상 흐름 6단계 순서 유지; 기준 미달은 기준 확인 후 데이터 부족 안내로 종료; 원인·설명 생성 오류는 해당 단계에서 재처리; 최근 24시간 고위험 신호는 안전 응답으로 종료	PASS 설계 일치
TP-02 KcELECTRA 분류	한국어 정서 문장 3개를 로컬 KcELECTRA 분류 모델에 입력(CPU)	모델 사용 가능; 감정 순서=[기쁨, 슬픔, 분노, 일반]; 출력 크기=문장 3개×감정 4종; 비정상 수치 없음; 문장별 확률합=[1.0, 1.0, 1.0]; 1.374초	PASS 출력 계약 충족

테스트 케이스	실행 방법	핵심 관측값	판정
TP-03 원문·Neo4j 수집	장기기억 수집 전용 테스트 3건과 실제 Neo4j 시험 데이터 저장·조회 실행	전용 테스트 3/3 통과(0.175초); 저장소 검사 6/6 통과; 사건 1건과 관련 정보 연결; 정리 후 시험 노드 0건	PASS 결합 경로 확인
TP-04 OpenAI LLM 연동	개인정보를 제거한 근거 1건을 실제 OpenAI API에 전달해 JSON 응답을 확인하고, 통제된 모의 응답으로 반복 검사	추천 1건 생성; 응답 형식 충족; 활동명·추천 이유·실천 방법 포함; 4.601초; 핵심 표현·원인·설명 단계의 근거 식별번호 제한 확인	PASS 연결·스키마· 파서 충족
TP-05 PostgreSQL 영속성	PostgreSQL 15.18 wellness_db에서 기존 데이터와 구분되는 시험 사용자로 저장·조회 시나리오 9개를 순서대로 실행	PostgreSQL 연결·안전한 SQL·최초 저장·같은 기간 갱신·중복 제거·JSON 재조회·데이터 부족 안내 교체·기간 조회·최신 리포트 조회 9/9 통과; 정리 후 시험 사용자·리포트 각 0건	PASS 영속성 정책 확인
TP-06 Tavily 연동	미리 준비한 정상 응답으로 Tavily 검색 조건·결과 정리와 검색 결과가 추천 생성으로 전달되는 과정 검사	검색 연동 검사 6/6 통과(API 주소·검색어·기본 검색·최대 3건·대기 5초·빈 본문 제외); 전달 흐름 검사 4/4 통과(검색 1회·추천 생성 1회·결과 1건·활동명 포함)	PASS 연동 로직 확인
TP-07 API·스케줄러· 안전	마음리포트 전체 Django 자동 테스트를 별도 테스트 데이터베이스에서 실행	발견·실행 78건; 18.698초; 전체 성공; 실패 0건; 오류 0건. 구성: 감정 계산·에이전트 흐름 49, API 12, 기간·예약 실행 7, Neo4j 3, 데이터 부족 안내 4, 안전 2, 제안 시점 1	PASS 전체 회귀 통과
TP-08 모듈화·그래프 구조	서비스 모듈 24개의 실행 가능 여부와 LangGraph 구성 조회	서비스 모듈 24/24 불러오기 성공; 실행 구조 10개 단계·19개 연결; 업무 단계 8개; 상태·에이전트 흐름·수집·감정 계산·원인·설명·검증·최종 응답·저장·실행 환경 파일 분리	PASS 모듈 경계 확인

6. 종합 결과

- 멀티에이전트 단계 전환·오류 재처리·데이터 부족 안내·안전 응답: PASS
- KcELECTRA 네 감정 분류와 확률 출력: PASS
- PostgreSQL 대화 원문과 Neo4j 장기기억 사건 결합: PASS
- OpenAI 생성형 LLM 연결과 JSON 응답 형식: PASS

- PostgreSQL 리포트 연속성 9/9: PASS
- Tavily 검색 연동 6/6·검색 결과 전달 흐름 4/4: PASS
- Django 전체 회귀 테스트 78/78: PASS
- 서비스 모듈 24/24 실행 가능·LangGraph 10개 단계/19개 연결: PASS

7. 결론

- 마음리포트 백엔드는 분석 기간의 사용자 대화 원문을 가장 중요한 근거로 사용하고, KcELECTRA의 감정 분류 결과와 Neo4j의 장기기억 사건을 보조 정보로 활용한다. 생성형 LLM은 실제로 제공된 원문과 장기기억 사건 안에서 핵심 표현·원인·설명·실천 제안을 만들며, 검증 단계에서 근거와 안전 조건을 다시 확인한다.

- 사전에 정의한 8개 테스트 계획을 실제 실행한 결과, 멀티에이전트 상태 전이, 모델 출력, 데이터 저장·조회, 외부 API 연동 모듈, API 스케줄러, 모듈화 검사가 모두 합격 기준을 충족하였다. 이에 따라 구현된 마음리포트 멀티에이전트 백엔드가 프로젝트 설계 의도에 맞게 동작함을 확인하였다.